RKNN2 SDK 快速入门

文件标识：RK-JC-YF-416

发布版本：V2.5.0

日期：2026-09-03

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档面向首次使用RKNN2 SDK的用户，目标是在最短路径内完成环境检查、模型转换、连板调试和板端部署。完整的软件栈说明、模型开发流程、API说明及系统化故障排查请参考对应的主文档。

读者对象

本文档主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本	修改人	修改日期	修改说明	核定人
V1.6.0	HPC	2023-11-28	初始版本	熊伟
V2.0.0-beta0	HPC	2024-03-15	增加RK3576内容	熊伟
V2.1.0	HPC	2024-08-01	使用Miniforge替换MiniConda	熊伟
V2.2.0	HPC	2024-09-04	更新版本号	熊伟
V2.3.0	HPC	2024-11-06	增加通过 pip 源安装 RKNN-Toolkit2 的支持	熊伟
V2.3.2	HPC	2025-04-03	增加 RV1126B 平台支持	熊伟
V2.5.0	HPC	2026-09-03	文档重构	熊伟

目录

1. 开始之前
    1.1 本指南目标
    1.2 软件框架与主要组件
    1.3 选择示例类型
    1.4 支持平台和版本要求
    1.5 获取SDK及配套仓库
2. 硬件连接与检查
    2.1 硬件清单
    2.2 开发板和连接工具介绍
    2.3 连接开发板
    2.4 检查设备连接状态
3. 主机端和板端环境准备
    3.1 安装RKNN-Toolkit2
        3.1.1 安装Python环境
        3.1.2 安装RKNN-Toolkit2
    3.2 检查Toolkit安装结果
    3.3 安装或检查板端RKNPU2环境
        3.3.1 确认NPU驱动版本
        3.3.2 检查rknn_server与Runtime库
        3.3.3 安装/更新RKNPU2环境
    3.4 安装编译工具
        3.4.1 安装CMake
        3.4.2 安装编译器
            3.4.2.1 确认开发板的系统类型、系统架构
            3.4.2.2 Android系统开发板安装NDK
            3.4.2.3 Linux系统开发板安装GCC交叉编译器
    3.5 运行环境预检查
4. 运行示例模型
    4.1 运行图像分类示例
        4.1.1 转换模型
        4.1.2 运行RKNN Python Demo
        4.1.3 运行RKNN C Demo
    4.2 运行目标检测示例
        4.2.1 转换模型

4.2.2 运行RKNN Python Demo
4.2.3 运行RKNN C Demo
4.2.4 数据集精度评估（可选）
5. Docker 中运行 RKNN Python Demo（可选）
5.1 安装Docker
5.2 准备RKNN-Toolkit2镜像
5.3 Docker中运行RKNN Python Demo
6. 常见启动问题与下一步
6.1 adb devices查看不到设备
6.2 手动启动rknn_server服务
6.3 后续文档导航

1. 开始之前

本指南以最短路径引导用户完成RKNN2 SDK的首次运行，涵盖硬件连接、环境准备、模型转换、连板调试和板端部署等最小必要步骤。模型转换原理、量化配置、完整API说明和系统化故障排查不在本指南范围内，请查阅对应文档。

> **说明：** 完整的开发流程、Toolkit接口说明、精度/性能/内存分析等，请参考《RKNN2 SDK 开发指南》文档。硬件、操作系统和版本支持范围以SDK发布包中的说明为准，本指南仅列出首次运行所需的必要信息。

1.1 本指南目标

完成本指南后，用户应能够：

1. 确认开发主机、开发板和RKNPU2运行环境可用；
2. 使用RKNN-Toolkit2完成一个ONNX模型的转换；
3. 通过连板调试方式在板端运行Python Demo；
4. 编译C Demo并在板端运行；
5. 根据文档导航进入完整开发、API参考或故障排查文档。

1.2 软件框架与主要组件

RKNN2 SDK的开发流程主要由PC端开发套件和开发板运行环境两部分组成：

- **PC端开发套件：** 用户可通过RKNN-Toolkit2将ONNX、PyTorch、TensorFlow等深度学习框架训练的模型转换为RKNN格式，并通过连板调试的方式在开发板上验证模型。RKNN Model Zoo提供模型转换与部署示例。
- **开发板运行环境：** 模型转换完成后，可在开发板上使用RKNPU2 Runtime加载和运行RKNN模型。

主要组件如下：

组件	运行位置	作用
`RKNN-Toolkit2`	PC端	模型转换、量化、连板调试等核心功能的Python工具包
`RKNN Server`（rknn_server）	板端	后台代理服务，用于接收PC传输的命令和数据，调用板端Runtime对应的接口执行，并将结果返回给PC
`RKNPU2 Runtime`（librknnrt.so）	板端	负责加载RKNN模型，并通过NPU执行RKNN模型的推理操作
`RKNN Model Zoo`	PC端/板端	提供 YOLOv5、YOLOv8、MobileNet 等常用模型的转换脚本和 Python/C++ 示例代码

1.3 选择示例类型

本指南使用以下两个示例贯穿首次运行流程：

示例	用途	Model Zoo目录
MobileNet	图像分类示例	`examples/mobilenet`
YOLOv5	目标检测示例	`examples/yolov5`

每个模型示例提供Python和C/C++两个版本的示例代码。为方便描述，后续分别使用RKNN Python Demo和RKNN C Demo表示。不同的Demo用法存在差异，请按照各自目录下README.md中的步骤运行。

1.4 支持平台和版本要求

支持的平台如下：

类别	说明
硬件平台	RK3562、RK3566系列、RK3568系列、RK3576系列、RK3588系列、RV1126B
开发主机	Ubuntu 18.04 / Ubuntu 20.04 / Ubuntu 22.04
Python版本	3.8（推荐版本）
RKNPU2驱动版本	建议 >= 0.9.8

> **注意：** 要保证rknn_server、Runtime库（librknnrt.so）与RKNN-Toolkit2三者的版本一致，建议都安装最新的版本。若使用动态维度输入的RKNN模型，则要求rknn_server和Runtime库版本 >= 1.5.0。

1.5 获取SDK及配套仓库

建议新建一个目录用来存放RKNN仓库，例如新建一个名称为Projects的文件夹，并将RKNN-Toolkit2和RKNN Model Zoo仓库存放至该目录下，参考命令如下：

```
# 新建 Projects 文件夹
mkdir Projects

# 进入该目录
cd Projects

# 下载 RKNN-Toolkit2 仓库
git clone https://github.com/airockchip/rknn-toolkit2.git --depth 1

# 下载 RKNN Model Zoo 仓库
git clone https://github.com/airockchip/rknn_model_zoo.git --depth 1

# 注意：
# 1.参数 --depth 1 表示只克隆最近一次 commit
# 2.如果遇到 git clone 失败的情况，也可以直接在 github 中下载压缩包到本地，然后解压至该目录
```

后续示例默认目录结构如下：

```
Projects
├── rknn-toolkit2
│   ├── doc
│   ├── rknn-toolkit2
│   │   ├── packages
│   │   ├── docker
│   │   └── ...
│   ├── rknpu2
│   │   ├── runtime
│   │   └── ...
│   └── ...
└── rknn_model_zoo
    ├── datasets
    ├── examples
    └── ...
```

2. 硬件连接与检查

本章只完成首次运行所需的硬件连接和设备可见性检查。

2.1 硬件清单

运行本文档示例程序需要准备以下硬件：

硬件	数量	说明
计算机	1	操作系统为Ubuntu 18.04/20.04/22.04
开发板	1	RK3562/RK3566/RK3568/RK3576/RK3588/RV1126B
数据线	1	类型随开发板而定，见2.2节
电源适配器	1	规格随开发板而定，见2.2节

> **说明：** 由于生产批次不同，开发板的数据线接口类型和位置可能会发生变化。通常情况下，开发板上印有OTG字样的接口为数据线接口。

2.2 开发板和连接工具介绍

1. 开发板

RK3562开发板：

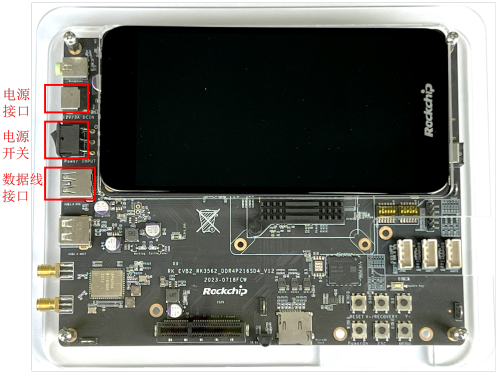

图2-1 RK3562开发板

RK3566开发板：

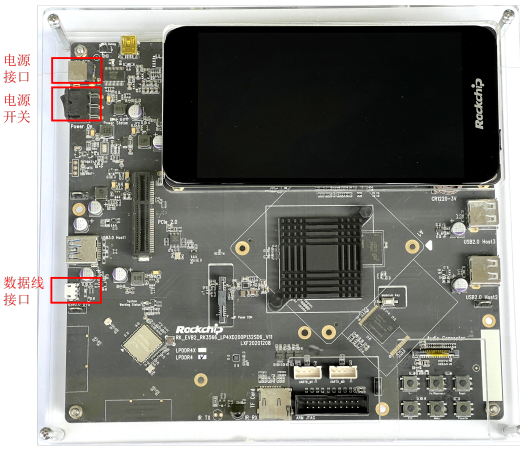

图2-2 RK3566开发板

RK3568开发板：

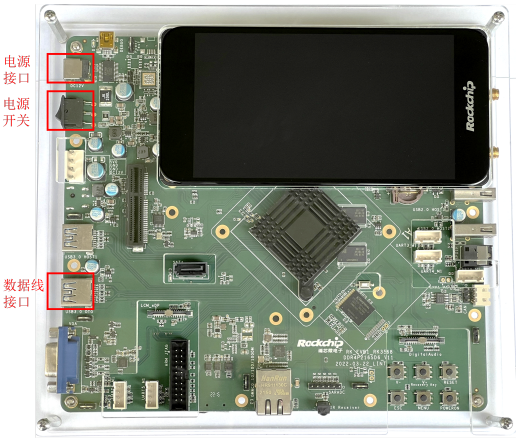

图2-3 RK3568开发板

RK3588开发板：

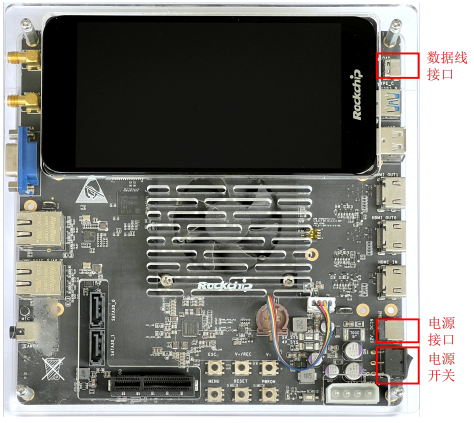

图2-4 RK3588开发板

RK3576开发板：

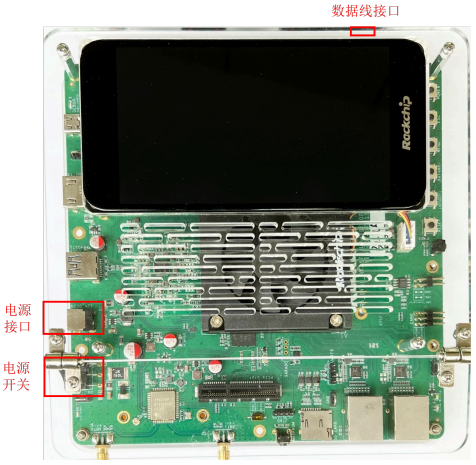

图2-5 RK3576开发板

RV1126B开发板：

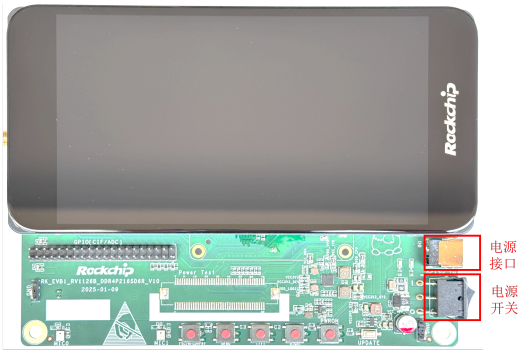

图2-6 RV1126B开发板

RV1126B开发板侧面：

图2-7 RV1126B开发板侧面

2. 数据线

USB-A数据线（适用于RK3562/RK3568开发板）：

图2-8 USB-A 数据线（适用于 RK3562/RK3568 开发板）

Micro USB数据线（适用于RK3566开发板）：

图2-9 Micro USB 数据线（适用于 RK3566 开发板）

USB-C数据线（适用于RK3576/RK3588开发板）：

图2-10 USB-C 数据线（适用于 RK3576/RK3588 开发板）

3. 电源适配器

输出12V-2A的电源适配器（适用于RK3562/RK3566/RK3568开发板）：

图2-11 输出 12V-2A 的电源适配器（适用于 RK3562/RK3566/RK3568 开发板）

输出12V-3A的电源适配器（适用于RK3576/RK3588开发板）：

图2-12 输出 12V-3A 的电源适配器（适用于 RK3576/RK3588 开发板）

2.3 连接开发板

下面以RK3568为例说明如何将开发板连至计算机：

1. 准备一台操作系统为Ubuntu 18.04 / Ubuntu 20.04 / Ubuntu 22.04的计算机。
2. 找到下图中电源接口的位置，连接开发板电源适配器。
3. 使用数据线连接开发板与计算机。（注意，由于生产批次不同，开发板的数据线接口类型和位置可能会发生变化。通常情况下，开发板上印有OTG字样的接口为数据线接口。）
4. 打开电源开关，等待开发板系统启动完成。
5. 查看开发板是否连接至计算机

在计算机的终端窗口（命令行界面）中，执行以下命令：

```
# 如果没有安装过 adb，请先使用 sudo apt install adb 安装
adb devices
```

连接成功时输出信息如下，其中`13af7b28115662cd`为RK3568的设备ID（示例值，实际以本机为准）。若无设备显示请参考6.1 adb devices查看不到设备章节进行排查。

```
List of devices attached
13af7b28115662cd    device
```

2.4 检查设备连接状态

在继续安装和模型转换前，至少确认以下检查全部通过：

检查项	检查命令	通过标准
ADB工具可用	`adb version`	能正常输出adb版本信息
板端设备可见	`adb devices`	设备列表中出现开发板设备，且状态为`device`

上述检查均通过后，再继续主机端和板端环境准备。

3. 主机端和板端环境准备

3.1 安装RKNN-Toolkit2

快速入门仅给出推荐的Conda + pip安装路径。Docker方式的安装请参考第 5 章。

3.1.1 安装Python环境

如果系统中没有安装**Python 3.8（建议版本）**，或者同时有多个版本的Python环境，建议使用Miniforge Conda创建新的Python 3.8环境。

步骤1：安装Miniforge Conda

在计算机的终端窗口中执行以下命令，检查是否安装Miniforge Conda，若已安装则可省略此步骤。

```
conda -V
# 参考输出信息：conda 23.3.1 ，表示 Miniforge conda 版本为 23.3.1
# 如果提示 conda: command not found，则表示未安装 Miniforge
```

如果没有安装Miniforge Conda，可以通过下面的链接下载Miniforge Conda安装包：
https://github.com/conda-forge/miniforge/releases/download/25.3.0-1/Miniforge3-25.3.0-1-Linux-x86_64.sh

然后通过以下命令安装Miniforge Conda：

```
# 拷贝到用户目录
cp /path/to/Miniforge3-25.3.0-1-Linux-x86_64.sh ~

# 增加运行权限
chmod 777 Miniforge3-25.3.0-1-Linux-x86_64.sh

# 运行安装脚本
bash Miniforge3-25.3.0-1-Linux-x86_64.sh
```

步骤2：创建Python环境

安装Miniforge Conda后，在计算机的终端窗口中，执行以下命令进入Conda base环境：

```
source ~/miniforge3/bin/activate # Miniforge 安装的目录
# 成功后，命令行提示符会变成以下形式：
# (base) xxx@xxx:~$
```

通过以下命令创建名称为toolkit2的Python 3.8环境：

```
conda create -n toolkit2 python=3.8
```

激活toolkit2环境，后续将在此环境中安装RKNN-Toolkit2：

```
conda activate toolkit2
# 成功后，命令行提示符会变成以下形式：
# (toolkit2) xxx@xxx:~$
```

> **说明：** 后续Python运行命令默认在`toolkit2`环境中执行。

3.1.2 安装RKNN-Toolkit2

激活toolkit2环境后，可通过pip源或者本地wheel包这两种方式安装RKNN-Toolkit2。

方式一：通过pip源安装

```
pip install rknn-toolkit2 -i https://pypi.org/simple

# 如果已安装 RKNN-Toolkit2，可通过以下命令升级 RKNN-Toolkit2
pip install rknn-toolkit2 -i https://pypi.org/simple --upgrade
```

方式二：通过本地wheel包安装

```
# 进入 rknn-toolkit2 目录
cd Projects/rknn-toolkit2/rknn-toolkit2

# 请根据不同的 python 版本及处理器架构，选择不同的 requirements 文件：
# 其中 cpxx 是 python 版本号
pip install -r packages/x86_64/requirements_cpxx.txt
# pip install -r packages/arm64/arm64_requirements_cpxx.txt

# 安装 RKNN-Toolkit2
# 请根据不同的 python 版本及处理器架构，选择不同的 wheel 安装包文件：
# 其中 x.x.x 是 RKNN-Toolkit2 版本号，cpxx 是 python 版本号
pip install packages/x86_64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
# pip install packages/arm64/rknn_toolkit2-x.x.x-cpxx-cpxx-
manylinux_2_17_aarch64.manylinux2014_aarch64.whl
```

3.2 检查Toolkit安装结果

保持`toolkit2`环境处于激活状态，执行：

```
python -c "from rknn.api import RKNN; print('RKNN-Toolkit2 import OK')"
```

若输出：

```
RKNN-Toolkit2 import OK
```

表示最基本的Toolkit导入检查通过。

如果安装失败，请查阅《RKNN2 SDK 开发指南》文档中的第5.4.2章节"工具安装问题"，其中详细介绍了RKNN-Toolkit2环境安装失败的解决方法。

3.3 安装或检查板端RKNPU2环境

板端运行Demo前，需要确认NPU驱动、rknn_server服务和Runtime库已正确安装。

3.3.1 确认NPU驱动版本

可以在板端执行以下命令查询RKNPU2驱动版本：

```
dmesg | grep -i rknpu
```

如下图所示，当前RKNPU2驱动版本为0.8.8。

```
rk3588_s_evb7:/ $ dmesg | grep -i rknpu
[    2.919448] [    T1] RKNPU fdab0000.npu: Adding to iommu group 0
[    2.919608] [    T1] RKNPU fdab0000.npu: RKNPU: rknpu iommu is enabled, using iommu mode
[    2.920935] [    T1] RKNPU fdab0000.npu: can't request region for resource [mem 0xfdab0000-0xfdabffff]
[    2.920959] [    T1] RKNPU fdab0000.npu: can't request region for resource [mem 0xfdac0000-0xfdacffff]
[    2.920973] [    T1] RKNPU fdab0000.npu: can't request region for resource [mem 0xfdad0000-0xfdadffff]
[    2.921339] [    T1] [drm] Initialized rknpu 0.8.8 20230428 for fdab0000.npu on minor 1
[    2.924802] [    T1] RKNPU fdab0000.npu: RKNPU: bin=0
[    2.924994] [    T1] RKNPU fdab0000.npu: leakage=10
[    2.925052] [    T1] debugfs: Directory 'fdab0000.npu-rknpu' with parent 'vdd_npu_s0' already present!
[    2.931816] [    T1] RKNPU fdab0000.npu: pvtm=866
[    2.936313] [    T1] RKNPU fdab0000.npu: pvtm-volt-sel=3
[    2.937408] [    T1] RKNPU fdab0000.npu: avs=0
[    2.937608] [    T1] RKNPU fdab0000.npu: l=10000 h=85000 hyst=5000 l_limit=0 h_limit=800000000 h_table=0
[    2.948777] [    T1] RKNPU fdab0000.npu: failed to find power_model node
[    2.948814] [    T1] RKNPU fdab0000.npu: RKNPU: failed to initialize power model
[    2.948826] [    T1] RKNPU fdab0000.npu: RKNPU: failed to get dynamic-coefficient
```

图3-1 RKNPU2 驱动版本信息

Rockchip开发板的官方固件均自带RKNPU2驱动。若以上命令查询不到NPU驱动版本，则可能使用的是第三方固件，其中可能没有安装NPU驱动。如果有固件源码，可以在`kernel config`中将`CONFIG_ROCKCHIP_RKNPU`选项的值改成y以集成NPU驱动，然后重新编译内核驱动并烧录。建议RKNPU2驱动版本 >= 0.9.8。

3.3.2 检查rknn_server与Runtime库

RKNN-Toolkit2的连板调试功能要求板端已安装RKNPU2环境，并且启动rknn_server服务。如果板端没有安装rknn_server和Runtime库，或者两者的版本不一致，都需要重新安装RKNPU2环境。

通常情况下，开发板默认已经安装版本一致的RKNPU2环境，可以通过下面命令确认（如果没有安装RKNPU2环境或者版本不一致，请按照3.3.3节中的步骤来安装/更新RKNPU2环境）。

1. 板端为Android系统

检查RKNPU2环境是否安装：如果能够启动rknn_server服务，则代表板端已经安装RKNPU2环境。

```
# 进入板端
adb shell

# 启动 rknn_server
su
setenforce 0
/vendor/bin/rknn_server &
```

如果出现以下输出信息，则代表启动rknn_server服务成功，即已经安装RKNPU2环境。

```
start rknn server, version: x.x.x
```

检查版本是否一致：

```
# 查询 rknn_server 版本
strings /vendor/bin/rknn_server | grep -i "rknn_server version"

# 查询 librknnrt.so 库版本
# 64位系统
strings /vendor/lib64/librknnrt.so | grep -i "librknnrt version"
# 32位系统
strings /vendor/lib/librknnrt.so | grep -i "librknnrt version"
```

如果出现以下输出信息，则代表rknn_server版本为x.x.x，librknnrt.so的版本为x.x.x。

```
rknn_server version: x.x.x
librknnrt version: x.x.x
```

2. 板端为Linux系统

检查RKNPU2环境是否安装：如果能够启动rknn_server服务，则代表板端已经安装RKNPU2环境。

```
# 进入板端
adb shell

# 启动 rknn_server
restart_rknn.sh
```

如果出现以下输出信息，则代表启动rknn_server服务成功，即已经安装RKNPU2环境。

```
start rknn server, version: x.x.x
```

检查版本是否一致：

```
# 查询 rknn_server 版本
strings /usr/bin/rknn_server | grep -i "rknn_server version"

# 查询 librknnrt.so 库版本
strings /usr/lib/librknnrt.so | grep -i "librknnrt version"
```

如果出现以下输出信息，则代表rknn_server版本为x.x.x，librknnrt.so的版本为x.x.x。

```
rknn_server version: x.x.x
librknnrt version: x.x.x
```

3.3.3 安装/更新RKNPU2环境

不同的板端系统需要安装不同的RKNPU2环境，下面分别介绍各自的安装方法。

> **说明：** 如果已经安装版本一致的RKNPU2环境，则本节内容可以跳过。

1. 板端为Android系统

进入rknpu2目录，使用adb工具将相应的rknn_server和librknnrt.so推送至板端，然后启动rknn_server，参考命令如下：

```
# 进入 rknpu2 目录
cd Projects/rknn-toolkit2/rknpu2

# 切换到 root 用户权限
adb root

# 挂载文件系统为可读写模式
adb remount

# 推送 rknn_server
# 注：在 64 位 Android 系统中，BOARD_ARCH 对应 arm64 目录；在 32 位系统中，对应 arm 目录。
adb push runtime/Android/rknn_server/${BOARD_ARCH}/rknn_server /vendor/bin/

# 推送 librknnrt.so
# 64位系统
adb push runtime/Android/librknn_api/arm64-v8a/librknnrt.so /vendor/lib64/
# 32位系统
adb push runtime/Android/librknn_api/armeabi-v7a/librknnrt.so /vendor/lib/

# 进入板端
adb shell

# 赋予可执行权限
chmod +x /vendor/bin/rknn_server

# 重启 rknn_server 服务
su
setenforce 0
/vendor/bin/rknn_server &
```

2. 板端为Linux系统

进入rknpu2目录，使用adb工具将相应的rknn_server和librknnrt.so推送至板端，然后启动rknn_server，参考命令如下：

```
# 进入 rknpu2 目录
cd Projects/rknn-toolkit2/rknpu2

# 推送 rknn_server 到板端
# 注：在64位Linux系统中，BOARD_ARCH对应aarch64目录，在32位系统，对应armhf目录。
adb push runtime/Linux/rknn_server/${BOARD_ARCH}/usr/bin/* /usr/bin

# 推送 librknnrt.so
adb push runtime/Linux/librknn_api/${BOARD_ARCH}/librknnrt.so /usr/lib

# 进入板端
adb shell

# 赋予可执行权限
chmod +x /usr/bin/rknn_server
chmod +x /usr/bin/start_rknn.sh
chmod +x /usr/bin/restart_rknn.sh

# 重启 rknn_server 服务
restart_rknn.sh
```

3.4 安装编译工具

3.4.1 安装CMake

在计算机的终端中，执行以下命令：

```
# 更新包列表
sudo apt update

# 安装 cmake
sudo apt install cmake
```

3.4.2 安装编译器

3.4.2.1 确认开发板的系统类型、系统架构

为了方便描述，后续文档使用"板端"来表示开发板端。

步骤1：确认板端系统类型

在计算机的终端中，执行以下命令：

```
adb shell getprop ro.build.version.release
```

如果输出是数字（Android系统版本号），则表示板端是Android系统：

```
12
```

否则板端是Linux系统：

```
/bin/sh: getprop: not_found
```

步骤2：确认板端系统架构

如果板端是Android系统，可以在计算机端执行以下命令查询系统架构：

```
adb shell getprop ro.product.cpu.abi
```

该命令的输出信息参考如下，其中arm64-v8a表示ARM 64位架构、第八版本的ABI。

```
arm64-v8a
```

如果板端是Linux系统，可以在计算机端执行以下命令查询系统架构：

```
adb shell uname -a
```

该命令的参考输出信息如下，其中aarch64表示ARM 64位架构。

```
Linux Rockchip 5.10.160 #183 SMP Tue Oct 24 18:52:11 CST 2023 aarch64 GNU/Linux
```

3.4.2.2 Android系统开发板安装NDK

> **适用范围：** 本节适用于Android系统的开发板，如果板端是Linux系统，请忽略此节。

- **NDK下载地址**（建议下载r19c版本）：https://dl.google.com/android/repository/android-ndk-r19c-linux-x86_64.zip
- **解压软件包**

建议将NDK软件包解压到Projects的文件夹中，位置如下：

```
Projects
├── rknn-toolkit2
├── rknn_model_zoo
└── android-ndk-r19c # 此路径在后面编译 RKNN C Demo 时会用到
```

此时，NDK编译器的路径就是`Projects/android-ndk-r19c`。

3.4.2.3 Linux系统开发板安装GCC交叉编译器

> **适用范围：** 本节适用于Linux系统的开发板，如果板端是Android系统，请忽略此节。

- **GCC下载地址**
- 板端为64位系统：https://developer.arm.com/-/cdn-downloads/permalink/legacy-linaro-gnu-toolchains/6.3-2017.05/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu.tar.xz
- 板端为32位系统：https://developer.arm.com/-/cdn-downloads/permalink/legacy-linaro-gnu-toolchains/6.3-2017.05/gcc-linaro-6.3.1-2017.05-x86_64_arm-linux-gnueabihf.tar.xz
- **解压软件包**

建议将GCC软件包解压到Projects的文件夹中。以板端为64位系统的GCC软件包为例，存放位置如下：

```
Projects
├── rknn-toolkit2
├── rknn_model_zoo
└── gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu # 此路径在后面编译RKNN C Demo时会用到
```

此时，GCC编译器的路径是`Projects/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu`。

3.5 运行环境预检查

在开始模型转换前，建议一次完成以下检查：

```
# Toolkit
conda activate toolkit2
python -c "from rknn.api import RKNN; print('Toolkit OK')"

# 编译工具
cmake --version

# 设备
adb devices
```

如果需要运行RKNN C Demo，还应根据板端系统确认编译器路径：

- 板端为Android系统：确认NDK路径（如`Projects/android-ndk-r19c`）；
- 板端为Linux系统：确认GCC编译器路径（如`Projects/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu`）。

4. 运行示例模型

本章以MobileNet（图像分类）和YOLOv5（目标检测）两个示例，演示"获取模型 -> 转换 -> 连板运行 -> 编译部署 -> 板端运行 -> 检查结果"的最小闭环。完整的模型转换配置、量化方法、精度/耗时/内存评估和API调用顺序请参考《RKNN2 SDK 开发指南》文档。

4.1 运行图像分类示例

本节以MobileNet模型为例，介绍图像分类模型的转换、连板运行和板端部署，示例位于`examples/mobilenet`目录。

4.1.1 转换模型

进入`rknn_model_zoo/examples/mobilenet/model`目录，运行download_model.sh脚本下载MobileNet ONNX模型：

```
# 进入 rknn_model_zoo/examples/mobilenet/model 目录
cd Projects/rknn_model_zoo/examples/mobilenet/model

# 下载 mobilenetv2-12.onnx 模型
./download_model.sh
```

进入`rknn_model_zoo/examples/mobilenet/python`目录，运行mobilenet.py脚本，该脚本将ONNX模型转成RKNN模型：

```
# 进入 rknn_model_zoo/examples/mobilenet/python 目录
cd Projects/rknn_model_zoo/examples/mobilenet/python

# 用法: python mobilenet.py --model <onnx_model> --target <TARGET_PLATFORM> [--dtype
<i8/fp>]
# 注: 这里以 rk3588 平台为例，如果是其他开发板，则需要修改命令中的平台类型
python mobilenet.py --model ../model/mobilenetv2-12.onnx --target rk3588
```

其中，`--dtype`省略时默认为`i8`（8位量化），`fp`表示不量化。该命令在转换完成后会直接在PC模拟器上进行推理，无需连接开发板；转换后的模型默认保存为`examples/mobilenet/model/mobilenet_v2.rknn`。

4.1.2 运行RKNN Python Demo

- **模拟器推理：** 不加`--npu_device_test`参数即可在PC模拟器上推理，命令同4.1.1节，适用于无板环境下的快速验证。
- **连板运行：** 追加`--npu_device_test`参数，通过连板调试的方式在板端推理：

```
python mobilenet.py --model ../model/mobilenetv2-12.onnx --target rk3588 --
npu_device_test
```

脚本运行成功后输出图像分类的TOP5结果，格式如下：

```
-----TOP 5-----
[494] score=0.990035 class=n03017168 chime, bell, gong
[469] score=0.003907 class=n02939185 caldron, cauldron
[653] score=0.001089 class=n03764736 milk can
[747] score=0.000945 class=n04023962 punching bag, punch bag, punching ball,
punchball
[442] score=0.000712 class=n02825657 bell cote, bell cot
```

其中，方括号内为类别索引，score为置信度，class为类别名称。

4.1.3 运行RKNN C Demo

完整运行一个RKNN C Demo，需要先将C/C++源代码编译成可执行文件，然后推送到板端，最后在板端运行。编译前需先安装编译工具（见3.4节）：板端为Android系统时在build-android.sh脚本开头配置ANDROID_NDK_PATH，板端为Linux系统时在build-linux.sh脚本开头配置GCC_COMPILER。编译脚本的参数说明见各示例目录下的README.md。

1. 编译

```
# 板端为 Android 系统
# 注：在 build-android.sh 脚本开头配置 ANDROID_NDK_PATH
cd Projects/rknn_model_zoo
./build-android.sh -t rk3588 -a arm64-v8a -d mobilenet

# 板端为 Linux 系统
# 注：在 build-linux.sh 脚本开头配置 GCC_COMPILER
cd Projects/rknn_model_zoo
./build-linux.sh -t rk3588 -a aarch64 -d mobilenet
```

2. 推送文件到板端

编译完成后，可执行文件位于`install`目录下的`rknn_mobilenet_demo`文件夹：

```
# 板端为 Android 系统
adb root
adb push install/rk3588_android_arm64-v8a/rknn_mobilenet_demo /data/

# 板端为 Linux 系统
adb push install/rk3588_linux_aarch64/rknn_mobilenet_demo /userdata/
```

3. 板端运行Demo

```
# 进入板端
adb shell

# 进入 rknn_mobilenet_demo 目录
# Android 系统为 /data/rknn_mobilenet_demo，Linux 系统为 /userdata/rknn_mobilenet_demo
cd /userdata/rknn_mobilenet_demo

# 设置依赖库环境
export LD_LIBRARY_PATH=./lib

# 运行可执行文件
# 用法: ./rknn_mobilenet_demo <model_path> <input_path>
./rknn_mobilenet_demo model/mobilenet_v2.rknn model/bell.jpg
```

输出结果同4.1.2节的TOP5格式。

4.2 运行目标检测示例

本节以YOLOv5模型为例，介绍目标检测模型的转换、连板运行、板端部署和精度评估，示例位于`examples/yolov5`目录。

4.2.1 转换模型

进入`rknn_model_zoo/examples/yolov5/model`目录，运行download_model.sh脚本，该脚本将下载一个可用的YOLOv5 ONNX模型，并存放在当前model目录下，参考命令如下：

```
# 进入 rknn_model_zoo/examples/yolov5/model 目录
cd Projects/rknn_model_zoo/examples/yolov5/model

# 运行 download_model.sh 脚本，下载 yolov5 onnx 模型
# 例如，下载好的 onnx 模型存放路径为 model/yolov5s_relu.onnx
./download_model.sh
```

进入`rknn_model_zoo/examples/yolov5/python`目录，运行convert.py脚本，该脚本将原始的ONNX模型转成RKNN模型，参考命令如下：

```
# 进入 rknn_model_zoo/examples/yolov5/python 目录
cd Projects/rknn_model_zoo/examples/yolov5/python

# 运行 convert.py 脚本，将原始的 ONNX 模型转成 RKNN 模型
# 用法: python convert.py model_path [rk3566|rk3588|rk3562] [i8/fp] [output_path]
python convert.py ../model/yolov5s_relu.onnx rk3588 i8 ../model/yolov5s_relu.rknn
```

其中，`i8`表示启用8位量化。转换完成后，模型目录中会生成对应的`.rknn`模型文件。

4.2.2 运行RKNN Python Demo

RKNN Python Demo通过连板调试的方式在板端运行YOLOv5模型。进入`rknn_model_zoo/examples/yolov5/python`目录，运行yolov5.py脚本：

```
# 运行 yolov5.py 脚本，在板端运行 yolov5 模型
# 用法: python yolov5.py --model_path {rknn_model} --target {target_platform} --
img_show

# 其中，如果带上 --img_show 参数，则会显示结果图片
# 注: 这里以 rk3588 平台为例，如果是其他开发板，则需要修改命令中的平台类型
python yolov5.py --model_path ../model/yolov5s_relu.rknn --target rk3588 --img_show

# 如果想先在计算机端运行原始的 onnx 模型，可以参考以下命令
# 用法: python yolov5.py --model_path {onnx_model} --img_show
python yolov5.py --model_path ../model/yolov5s_relu.onnx --img_show
```

脚本运行成功后输出信息如下，其中，class字段代表预测的类别，score为得分，(xmin, ymin)是检测框的左上角坐标，(xmax, ymax)是检测框的右下角坐标。

```
# class @ (xmin, ymin, xmax, ymax) score
person @ (209 244 286 506) 0.884
person @ (478 238 559 526) 0.868
person @ (110 238 230 534) 0.825
bus @ (94 129 553 468) 0.705
person @ (79 354 122 516) 0.339
```

默认输入图片是`model/bus.jpg`，结果图片如下所示：

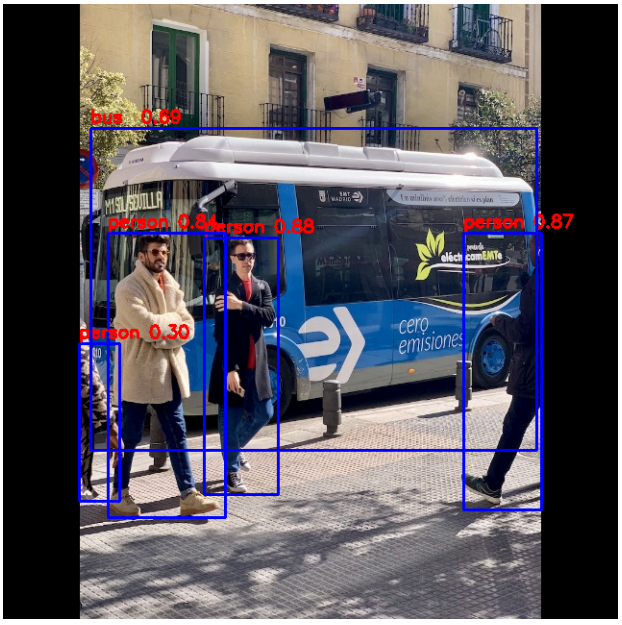

图4-1 RKNN Python Demo 可视化结果

4.2.3 运行RKNN C Demo

完整运行一个RKNN C Demo，需要先将C/C++源代码编译成可执行文件，然后推送到板端，最后在板端运行。编译前需先安装编译工具（见3.4节）：板端为Android系统时在build-android.sh脚本开头配置ANDROID_NDK_PATH，板端为Linux系统时在build-linux.sh脚本开头配置GCC_COMPILER。编译脚本的参数说明见各示例目录下的README.md。

1. 编译

```
# 板端为 Android 系统
# 注：在 build-android.sh 脚本开头配置 ANDROID_NDK_PATH
cd Projects/rknn_model_zoo
./build-android.sh -t rk3588 -a arm64-v8a -d yolov5

# 板端为 Linux 系统
# 注：在 build-linux.sh 脚本开头配置 GCC_COMPILER
cd Projects/rknn_model_zoo
./build-linux.sh -t rk3588 -a aarch64 -d yolov5
```

2. 推送文件到板端

编译完成后，可执行文件位于`install`目录下的`rknn_yolov5_demo`文件夹：

```
# 板端为 Android 系统
adb root
adb push install/rk3588_android_arm64-v8a/rknn_yolov5_demo /data/

# 板端为 Linux 系统
adb push install/rk3588_linux_aarch64/rknn_yolov5_demo /userdata/
```

3. 板端运行Demo

```
# 进入板端
adb shell

# 进入 rknn_yolov5_demo 目录
# Android 系统为 /data/rknn_yolov5_demo，Linux 系统为 /userdata/rknn_yolov5_demo
cd /userdata/rknn_yolov5_demo

# 设置依赖库环境
export LD_LIBRARY_PATH=./lib

# 运行可执行文件
# 用法: ./rknn_yolov5_demo <model_path> <input_path>
./rknn_yolov5_demo model/yolov5s_relu.rknn model/bus.jpg
```

默认情况下，输出图片保存路径为`rknn_yolov5_demo/out.png`，可以通过adb工具从板端拉取到本地查看，在本地计算机的终端中，执行以下命令：

```
# 拉取到本地当前目录
adb pull /userdata/rknn_yolov5_demo/out.png .
```

输出结果图片如下所示：

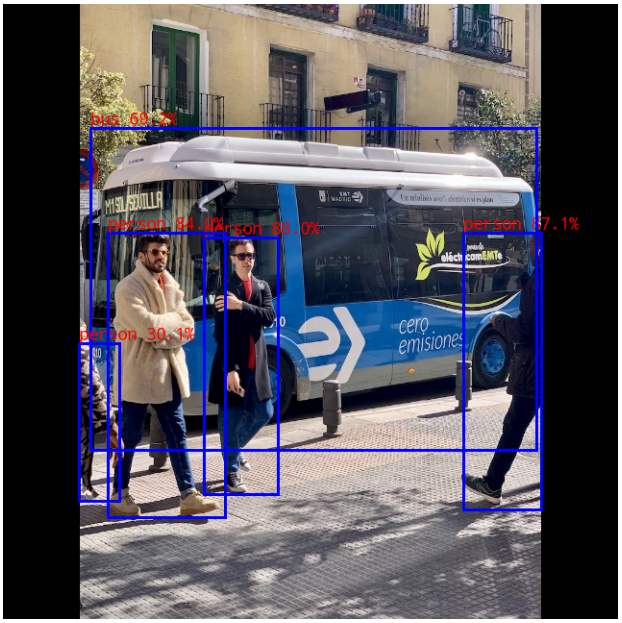

图4-2 RKNN C Demo 可视化结果

首次运行只需确认Demo正常结束并生成`out.png`。实际检测结果会随模型版本、输入图片及运行环境变化。

4.2.4 数据集精度评估（可选）

`rknn_model_zoo/datasets`目录存放数据集，用于精度评估，需要先下载评估数据集并保存至该目录。例如，对于YOLOv5模型，需要下载COCO数据集。进入`rknn_model_zoo/datasets/COCO`目录，运行download_eval_dataset.py脚本，该脚本将下载val2017数据集，并存放在当前COCO目录下，参考命令如下：

```
# 进入 rknn_model_zoo/datasets/COCO 目录
cd Projects/rknn_model_zoo/datasets/COCO

# 运行 download_eval_dataset.py 脚本，下载 COCO 数据集
python download_eval_dataset.py
```

进行数据集精度评估时，需要指定`--coco_map_test`参数，并指定评估数据集路径`--img_folder`，参考命令如下：

```
# 请先安装 pycocotools
pip install pycocotools

# 进入 rknn_model_zoo/examples/yolov5/python 目录
cd Projects/rknn_model_zoo/examples/yolov5/python

# 运行 yolov5.py 脚本
python yolov5.py \
    --model_path ../model/yolov5s_relu.rknn \
    --target rk3588 \
    --img_folder ../../../datasets/COCO/val2017 \
    --coco_map_test
```

> **说明：** 模型级精度分析（逐层对比量化模型与浮点模型的推理差异）请使用RKNN-Toolkit2的`rknn.accuracy_analysis()`接口，详细说明参考《RKNN2 SDK 开发指南》文档中的"模型精度分析"章节。

5. Docker 中运行 RKNN Python Demo（可选）

如果需要在Docker环境中运行RKNN Python Demo，可以参考本章中的内容准备开发环境。

请特别注意，这里提供了一个包含RKNN-Toolkit2环境的Docker镜像，允许用户在其中直接运行RKNN Python Demo而无需担心环境安装问题。但是，该Docker镜像中只包含纯净的RKNN-Toolkit2环境，仅适用于运行RKNN Python Demo。

5.1 安装Docker

如果已安装Docker，可跳过此步骤。如果没有安装，请根据官方手册进行安装。

Docker安装官方手册链接：https://docs.docker.com/install/linux/docker-ce/ubuntu/

注意事项：需要将用户添加到docker用户组。

```
# 创建docker用户组
sudo groupadd docker

# 把当前用户加入docker用户组
sudo usermod -aG docker $USER

# 更新激活docker用户组
newgrp docker

# 验证不需要sudo执行docker命令
docker run hello-world
```

成功安装的参考输出信息如下：

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
719385e32844: Pull complete
Digest: sha256:88ec0acaa3ec199d3b7eaf73588f4518c25f9d34f58ce9a0df68429c5af48e8d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
```

5.2 准备RKNN-Toolkit2镜像

本节介绍两种创建RKNN-Toolkit2镜像环境的方式，可任选一种方式进行创建。

1. 通过Dockerfile创建RKNN-Toolkit2镜像

在RKNN-Toolkit2工程中`docker/docker_file`文件夹下，提供了构建RKNN-Toolkit2开发环境的Dockerfile文件，用户通过docker build命令创建镜像，如下所示：

```
# 注：以下 xx 和 x.x.x 代表版本号，请根据实际数值进行替换
cd Projects/rknn-toolkit2/rknn-toolkit2/docker/docker_file/ubuntu_xx_xx_cpxx
docker build -f Dockerfile_ubuntu_xx_xx_for_cpxx -t rknn-toolkit2:x.x.x-cpxx .
```

2. 通过加载已打包的Docker镜像文件创建RKNN-Toolkit2镜像

通过如下链接下载对应版本的RKNN-Toolkit2工程文件，解压后在`docker/docker_image`文件夹下提供了已打包所有开发环境的Docker镜像。

Docker镜像文件网盘下载链接：https://console.zbox.filez.com/l/I00fc3 （提取码：rknn）

执行以下命令加载对应Python版本的镜像文件。

```
# 注：x.x.x 代表 RKNN-Toolkit2 的版本号，cpxx 代表的是 Python 的版本
docker load --input rknn-toolkit2-x.x.x-cpxx-docker.tar.gz
```

创建或加载镜像成功后，可以通过以下命令查看docker的镜像信息。

```
docker images
```

相应的RKNN-Toolkit2镜像信息显示如下：

```
REPOSITORY    TAG        IMAGE ID     CREATED     SIZE
rknn-toolkit2 x.x.x-cpxx xxxxxxxxxxxx 1 hours ago 5.89GB
```

5.3 Docker中运行RKNN Python Demo

> **注意：** 请确保板端已经安装RKNPU2环境，如果没有，请参考3.3章节中的内容进行安装。

1. 映射文件，并运行容器

请参考1.5章节在本地下载好RKNN Model Zoo工程，然后将其映射进容器中，并通过docker run命令运行容器。运行后将进入容器的bash环境。参考命令如下：

```
# 使用 docker run 命令创建并运行 RKNN Toolkit2 容器
# 并通过附加 -v <host src folder>:<image dst folder> 参数，将本地文件映射进容器中
docker run -t -i --privileged \
    -v /dev/bus/usb:/dev/bus/usb \
    -v /Projects/rknn_model_zoo:/rknn_model_zoo \
    rknn-toolkit2:x.x.x-cpxx \
    /bin/bash
```

2. 在容器中准备模型并转换

进入容器后，按照4.2.1章节的步骤，在容器内完成YOLOv5模型的下载和转换。

3. 在容器中运行RKNN Python Demo

按照4.2.2章节的步骤运行RKNN Python Demo，参考命令如下：

```
# 进入 rknn_model_zoo/examples/yolov5/python 目录
cd rknn_model_zoo/examples/yolov5/python

# 运行 yolov5.py 脚本，在板端运行 yolov5 模型
# 用法: python yolov5.py --model_path {rknn_model} --target {target_platform}
# 注: 这里以 rk3588 平台为例，如果是其他开发板，则需要修改命令中的平台类型
python yolov5.py --model_path ../model/yolov5s_relu.rknn --target rk3588
```

> **注意：** 当使用USB连接开发板时，请确保本地计算机和Docker容器中同时只开启一个adb server服务。如果需要在Docker容器中连接开发板，请在计算机的终端中执行命令`adb kill-server`终止本地计算机上的adb server服务。

6. 常见启动问题与下一步

本章收录首次运行过程中可能遇到的问题及排查方法。完整的FAQ、模型转换问题、精度问题、性能问题和内存问题请参考《RKNN2 SDK 开发指南》文档第10章"常见问题"。

6.1 adb devices查看不到设备

如果`adb devices`没有设备，可以尝试以下方式来解决此问题：

1. 检查连线是否正确、重新插拔数据线、换计算机另一个USB端口来连接数据线、更换数据线。
2. 当使用USB连接开发板时，请确保本地计算机和Docker容器中同时只开启一个adb server服务。例如，如果需要在Docker容器中连接开发板，请在计算机的终端中执行命令`adb kill-server`终止本地计算机上的adb server服务。
3. 出现以下错误时，表示系统中未安装adb。需要执行安装命令`sudo apt install adb`安装adb。

```
command 'adb' not found, but can be installed with:
sudo apt install adb
```

6.2 手动启动rknn_server服务

RKNN-Toolkit2的连板调试功能要求板端已安装RKNPU2环境，并且启动rknn_server服务。

但有些开发板系统没有开机自启动rknn_server服务，这导致在运行RKNN Python Demo时会遇到如`E init_runtime: The rknn_server on the concected device is abnormal.`的报错信息。此时请参考3.3.2章节中的相关内容手动启动rknn_server服务。

6.3 后续文档导航

完成首次运行后，请根据任务进入对应的主文档，避免在快速入门中继续查找完整API或高级配置。

需要继续了解的内容	主文档/入口
RKNN-Toolkit2完整安装、模型转换、量化配置	《RKNN2 SDK 开发指南》
Toolkit方法、参数和返回值（Python API）	《RKNN2 SDK 开发指南》
Runtime C API说明	《RKNN2 Runtime C API参考》
工具安装/NPU环境/模型转换/模型评估常见问题	《RKNN2 SDK 开发指南》
模型精度分析（accuracy_analysis）	《RKNN2 SDK 开发指南》
Docker方式安装RKNN-Toolkit2	《RKNN2 SDK 开发指南》
RKNN Model Zoo各模型示例用法	`rknn_model_zoo/examples/<模型名>/README.md`